DW-Bench: Benchmarking LLMs on Data Warehouse Graph Topology Reasoning

Ahmed G.A.H Ahmed
Innosol / Bahçeşehir University
ahmed.gamal@innosol-labs.eu

C. Okan Sakar
Department of Computer Engineering, Bahcesehir University, Turkey
okan.sakar@bau.edu.tr

Abstract

Text-to-SQL benchmarks test whether a model can write the right query. They do not test whether it understands the schema it queries. A data warehouse schema is a graph: tables linked by foreign keys, rows anchored to source records through ETL lineage. Tracing that structure (finding join paths, detecting disconnected silos, propagating impact through lineage chains) is routine data engineering, but no existing benchmark measures it.

We present **DW-Bench**, a benchmark for graph topology reasoning over data warehouse schemas. It contains 1,046 questions (13 subtypes, three difficulty levels, five datasets, 262 tables). We run six baselines, from flat context injection to tool-calling and code execution, against Gemini 2.5 Flash, DeepSeek-V3, and Qwen2.5-72B.

Tool-augmented baselines reach 87–90% micro-EM; static methods top out at 63–81%. The gap narrows on easy subtypes and widens on hard ones: no method exceeds 61% while the oracle clears $\geq$99.5%. That gap is a reasoning problem. Obfuscating table names drops static baselines 9–32 pp; tool-use loses under 4 pp.

Code, data, and per-question results: `https://github.com/AJamal27891/dw-bench`.

1 Introduction

Text-to-SQL has made rapid progress: state-of-the-art LLMs now approach or match human-level accuracy on Spider [21] and BIRD [13], which measure a model's capacity to translate a natural-language question into a SQL query. However, even the latest benchmarks like Spider 2.0 [11] show that enterprise-scale SQLworkflows remain largely unsolved (best models $\sim$17–21%).

Data warehouses, on the other hand, present a different kind of problem. A data engineer maintaining an enterprise warehouse does not primarily write SQL. Instead, they trace graph structure: lineage edges, foreign key (FK) paths, disconnected schema silos. Common questions include:

- *"If the `Person` table changes, which downstream tables are affected through data lineage?"* (impact analysis)
- *"Which tables form disconnected silos in our schema graph?"* (connectivity analysis)
- *"What is the shortest FK path between two tables?"* (routing)

Answering these questions demands breadth-first search (BFS), connected-component detection, and multi-hop path enumeration. No existing benchmark assesses LLMs on these capabilities over real-world data warehouse schemas with heterogeneous edge semantics.

Our evaluation reveals three principal findings: (i) all models achieve near-perfect scores on single-hop structural queries but drop 30–40 percentage points on compositional multi-hop tasks, exposing a systematic *structural reasoning ceiling*; (ii) Tool-Use prompting closes the gap on topology-enumeration subtypes but fails on compositional lineage-impact queries that require chaining multiple graph algorithms; and (iii) obfuscating table names degrades performance by up to 32 pp on lineage tasks, confirming that models partially rely on lexical cues rather than genuine graph traversal.

Our contributions are summarized as follows:

1. **DW-Bench**: 1,046 schema-level questions across 5 data warehouse schemas (4 real-world, 1 synthetic), covering 3 categories (lineage impact, schema routing, silo detection), 13 subtypes, and 3 difficulty levels.
2. **Six baselines** (Flat Text, Vector-RAG, Graph-Augmented, Tool-Use, ReAct-Code, Oracle), tested with Gemini 2.5 Flash, DeepSeek-V3, and Qwen2.5-72B.
3. **An obfuscation protocol** that randomizes table names, enabling controlled evaluation of memorization versus genuine topology understanding.
4. **Syn-Logistics**: A synthetic dataset with $n \geq 20$ per subtype. It exposes a model-dependent structural gap: Gemini scores 93–100% on `hop_count/count`, while DeepSeek drops to 27–48% on the same subtypes.
5. **Open-source release** of all datasets, evaluation code, and baseline implementations.

2 Related Work

Text-to-SQL Benchmarks. Spider [21] established the standard benchmark setting, with 10,181 questions spanning 200 databases. BIRD [13] introduced noisier and more realistic schemas, while Spider 2.0 [11] scaled the setting toward enterprise-style schemas. ScienceBenchmark focuses on scientific databases. RobuT [23] evaluates cell-level and row-level reasoning. All of these benchmarks target *query generation or cell-level reasoning*; none explicitly evaluate structural understanding.

Schema Understanding. BEAVER [1] evaluates LLMs on database knowledge including schema matching and entity resolution, but does not evaluate graph topology reasoning. LLM-FK [18] evaluates FK discovery, a single-hop structural task complementary to DW-Bench's multi-hop topology reasoning.

Graph Reasoning in LLMs. NLGraph [20] and GraphQA [4] evaluate LLMs on synthetic graph problems (connectivity, shortest path). GraCoRe [22] taxonomizes graph reasoning; GraphArena [17] benchmarks graph algorithms at scale. However, none of these use real database schemas, and none combine FK and lineage edge types.

Graph-Centric Agents and GNN Hybrids. Think-on-Graph [16] interleaves LLM reasoning with knowledge graph exploration; Graph Chain-of-Thought [9] augments prompts with graph-derived reasoning chains. G-Retriever [7] uses GNN encoders to retrieve the graph context for LLM QA. Retrieval-augmented generation [12] (RAG) and hierarchical GraphRAG [3] address knowledge-intensive tasks but do not specifically target schema topology. Recent work on tool-based graph reasoning agents (e.g., GraphWalk) explicitly studies scale and iteration budgets for compositional navigation, while structural-reward training frameworks (e.g., STRuCT-LLM) offer alternative methods to scaffold compositional reasoning. DW-Bench does not compare against these advanced graph-agent frameworks; following the precedent of Spider and BIRD, we evaluate standardized, zero-shot baselines to establish a *lower bound*. These methods are natural next steps: the Oracle upper bound ($\geq$99.5% EM) quantifies exactly how much room remains for hybrid architectures combining tools with learned graph representations.

Table 1: DW-Bench dataset statistics. "Silos" = disconnected components in the combined FK+lineage graph.

Dataset	Domain	Tbl	FK	Lin.	Qs	Silos
AdventureWorks	Retail	102	136	39	208	11
TPC-DS	Analytics	24	70	0	127	1
TPC-DI	ETL	35	29	21	181	2
OMOP CDM	Health	37	74	21	158	3
Syn-Logistics	Supply	64	96	35	372	5
Total		**262**	**405**	**116**	**1046**	—

Positioning. DW-Bench uniquely combines: (1) *real-world* schemas from industry standards, (2) *heterogeneous* edge types (FK + lineage), (3) *practical* question types from data engineering workflows, and (4) *obfuscation* for contamination control.

3 DW-Bench: Benchmark Design

3.1 Datasets

We select five datasets shown in Table 1 representing diverse data warehouse topologies:

AdventureWorks is a Microsoft reference data warehouse with 102 tables spanning OLTP and DW layers, connected by 136 FK edges and 39 lineage (derived_from) edges. Its dual-layer structure makes it ideal for lineage impact questions.

TPC-DS is the industry standard analytics benchmark with a star schema of 24 tables and 70 FK edges. Its single connected component and absence of lineage edges tests pure FK-based reasoning.

TPC-DI is the data integration benchmark with 35 tables modeling an ETL pipeline from staging to warehouse, with 21 lineage edges representing the transformation flow.

OMOP CDM (Observational Medical Outcomes Partnership Common Data Model) [8] is a healthcare standard with 37 tables, 74 FK edges, and 21 lineage edges across 3 connected components (clinical data, vocabulary tables, and metadata).

Syn-Logistics is our own synthetic supply-chain schema: 64 tables, 5 connected components (carrier, procurement, healthcare, finance, HR). We built it to fix a statistical-power problem in the real-world corpus: every subtype gets $n \geq 20$ questions. Table names come from a domain dictionary so obfuscation does not collapse semantics.

3.2 Schema Graph Representation

Each dataset is represented as a heterogeneous graph $G = (V, E_{fk} \cup E_{lin})$, where V denotes the set of table nodes, E_{fk} denotes the set of foreign-key edges, and E_{lin} denotes the set of data-lineage edges, using PyTorch Geometric [5] `HeteroData`. Each node represents a table and is associated with six structural features: in-degree, out-degree, normalized degree, lineage degree, betweenness centrality, and PageRank. Edges are typed as either `fk_to` (foreign key) or `derived_from` (data lineage). Lineage edges form a strict DAG (directed acyclic graph): each `derived_from` edge points from a downstream DW table to its upstream source, with no cycles or self-references. All traversal algorithms respect edge typing: FK queries use only `fk_to` edges, lineage queries use only `derived_from`, and `combined_impact` queries compose both types sequentially. FK edges are treated as **directed** (parent→child) for path and hop-count queries (`join_path`, `hop_count`, `direct_fk`) and as **undirected** for connectivity queries (`membership`, `connected`, `isolation`, `count`, `full_enum`).Lineage edges are always directed (downstream→upstream), following the `derived_from` convention defined above. Foreign keys are treated as bidirectional (undirected) explicitly for connectivity and silo-detection subtypes, as an FK implies an inherent structural relationship that can be traversed logically in either direction (e.g., from a parent lookup table to child fact rows, or vice versa).

Table 2: Question taxonomy. Difficulty is assigned based on the number of reasoning hops and edge types required.

Category	Subtype	Diff.	Description	#Qs
Lin.	forward	Easy	Direct lineage targets	72
	reverse	Easy	Source tables for DW table	71
	multi_source	Med.	Tables with 3+ sources	49
	transitive	Hard	Multi-hop lineage chains	33
	combined_impact	Hard	Lineage + FK dependents	69
Rt.	direct_fk	Easy	FK adjacency check	100
	join_path	Med.	FK shortest path	342
	hop_count	Med.	Path length	124
Silo	count	Easy	Number of components	30
	isolation	Med.	Is table X isolated?	60
	connected	Med.	Are X and Y connected?	29
	membership	Hard	Which component has X?	38
	full_enum.	Hard	List all tables in silo	29

For path-based questions, we validate alternative valid shortest paths against the FK adjacency matrix, ensuring EM is robust to tie-breaking.

3.3 Question Generation

Questions are generated deterministically from the graph structure using standard graph algorithms (BFS, connected component detection) implemented with NetworkX [6], ensuring ground-truth answers are provably correct. We define three categories with 13 subtypes as shown in Table 2.

Difficulty assignment. Difficulty labels are assigned a priori based on structural complexity, following the precedent of Spider [21] (which uses SQL AST depth) and BIRD [13]. Easy subtypes require a single graph lookup (one hop, one edge type). Medium subtypes require multi-hop traversal within a single edge type. Hard subtypes require either multi-hop transitive closure or composition across both FK and lineage edge types. These labels are fixed properties of the question structure, independent of any model's empirical performance.

3.4 Obfuscation Protocol

To distinguish genuine topology-based reasoning from reliance on surface lexical cues and schema-name memorization, we generate an *obfuscated* variant for each dataset. Table names are replaced with random identifiers (`Table_A`, `Table_B`, etc.) using a deterministic mapping. This prevents models from exploiting semantic signals in names such as `Customer` or `Invoice` to infer relationships without genuine structural reasoning. Questions and answers are updated accordingly with word-boundary-aware replacement to avoid corrupting natural language phrases (e.g., the word "relationship" in "foreign key relationship" is preserved).

4 Baselines

Six baselines are evaluated under a shared prompt with different context injection strategies, all in the zero-shot setting. Each paradigm receives the form of context that is natural to its architecture, rather than being constrained to token-budget parity, since the goal is to compare *reasoning paradigms* rather than token counts. The baseline methods are briefly summarized below.

Flat Text (FT). Dump the whole schema (tables, columns, FKs, lineage) as plain text. This mirrors what a developer does when pasting data definition language (DDL) into a long-context model.

Table 3: Main results (EM % ± 95% CI, pooled across 5 datasets, 2000 bootstrap resamples). **Bold** = best non-oracle.

Model		FT	VR	GA	TU	RC
Gem.	Micro	76.6	72.9	75.4	**89.3**	81.4
	Macro	77.0	72.4	77.1	**86.6**	69.9
DS	Micro	69.5	71.0	76.8	**90.4**	79.4
	Macro	68.2	62.4	69.5	**88.4**	71.6
Qw.	Micro	63.2	69.5	80.9	**87.5**	64.6
	Macro	64.8	62.8	77.5	**82.5**	62.2
Oracle			99.5 / 99.8 / 100.0			

Vector-RAG (VR). Embed schema elements with Sentence-BERT [15], retrieve k=15 chunks via FAISS [10], inject as context.

Graph-Augmented (GA). Extract a 3-hop BFS neighborhood around the tables mentioned in the question. For global questions with no anchor tables ("how many components?"), supply the full graph instead.

Tool-Use (TU). Provide the model with nine graph reasoning tools: `shortest_path`, `connected_components`, `get_component_of`, `get_fk_neighbors`, `get_lineage_forward`, `get_lineage_reverse`, `transitive_lineage`, `check_fk_adjacency`, `list_tables`. A maximum of three tool calls is allowed per question.

ReAct-Code (RC). The model writes and runs Python/NetworkX code in a sandboxed REPL, up to 5 rounds. More flexible than TU and serves as a budget ablation (5 code rounds vs. TU's 3 tool calls).

Oracle. Gold algorithmic outputs are injected into the prompt. This establishes the upper bound.

5 Experiments

5.1 Setup

We evaluate with **Gemini 2.5 Flash** [19] (closed frontier), **DeepSeek-V3** [2] (671B MoE, open-weight), and **Qwen2.5-72B** [14] (72B dense, open-weight), using greedy decoding (τ=0) for deterministic reproducibility. We report pooled **Micro-EM** (overall exact match) and **Macro-EM** (mean of per-subtype EM, weighting all 13 subtypes equally). For list-typed topology tasks (e.g., connected component membership), we apply **target-node normalization** (stripping the queried table from both prediction and gold prior to scoring) ensuring models are not penalized for harmless self-inclusion. An **Oracle** baseline injects gold algorithmic outputs, establishing an upper bound ($\geq$99.5% EM across all three models; the residual gap reflects minor formatting, not reasoning failures).

5.2 Main Results

Results are shown in Table 3 and the main findings are summarized below:

Finding 1: Agentic baselines dominate. TU tops the ranking at 87–90% micro-EM, roughly 7–14 pp ahead of the best static baseline. RC also clears static methods through code generation.

Finding 2: Triviality Illusion. FT looks competitive at 76.6% micro-EM, but the illusion comes from `join_path` (33% of questions) inflating the average. Macro-EM, which weights subtypes equally, puts GA ahead of FT among static methods; all still trail TU by >8 pp (Appendix Figure 3).

Table 4: EM (%) by difficulty. TU achieves near-perfect easy scores but plateaus on hard questions alongside static baselines.

Model	Diff.	FT	VR	GA	TU	RC
Gem.	Easy	82.7	82.5	82.2	**99.6**	90.4
	Med.	78.9	73.0	76.3	**90.3**	78.7
	Hard	56.1	46.5	55.6	59.9	**60.9**
DS	Easy	82.7	84.4	87.1	**98.0**	88.8
	Med.	65.3	65.3	76.0	**97.0**	76.3
	Hard	40.1	43.6	50.0	**60.4**	58.9
Qw.	Easy	77.4	84.0	86.6	**96.1**	72.2
	Med.	57.7	63.0	90.7	**92.0**	64.7
	Hard	33.2	40.1	51.0	**57.4**	44.1
Oracle Hard		99.0 / 99.5 / 100.0				

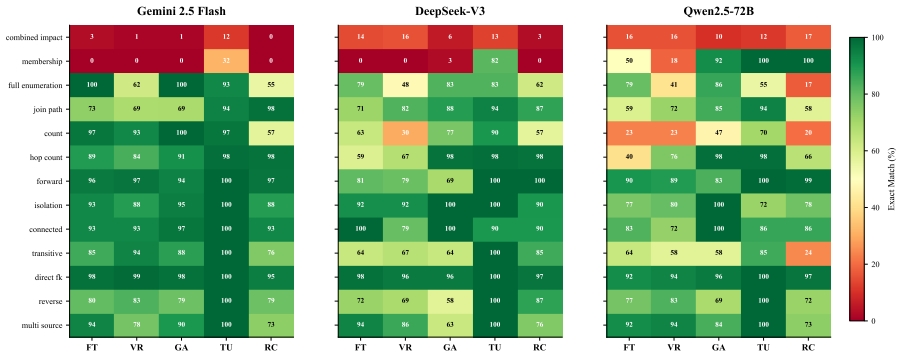

Figure 1: EM (%) by subtype for all three models. With target-node normalization, TU now averages 71% on `membership` (with Qwen reaching 100%), indicating that topology enumeration is fully solvable with tools. `combined_impact` remains at 0–17%, isolating compositional multi-hop reasoning as the core bottleneck. `full_enumeration` scores reach up to 100% (Gemini) for FT/GA because these questions are answered from the full schema context that Flat Text and Graph-Augmented inject by default for global graph queries.

5.3 Results by Difficulty

Easy/Medium patterns. As seen in Table 4, Gemini FT scores 78.9% on Medium versus 82.7% on Easy. Medium is dominated by `join_path` (342 of 544 Medium questions), which has high FT accuracy since short FK paths are often present in the flat schema text. Easy includes subtypes like `count` (number of connected components) that require global graph access unavailable to FT. The distribution reflects subtype composition, not a flaw in difficulty labels.

5.4 Per-Subtype Analysis

Finding 3: Hard-task ceiling. The results based on subtype analysis is shown in Figure 1. All baselines plateau at $\sim$60% on hard questions while Oracle achieves $>$98%. The `combined_impact` subtype ($n = 69$) averages only 12% EM even with TU, identifying compositional graph reasoning as the fundamental bottleneck. RC (5 code rounds) shows a similar ceiling to TU (3 tool calls), suggesting the bottleneck is compositional reasoning itself rather than interaction budget.

Table 5: Obfuscation penalty (4 datasets). TU is nearly invariant.

Model		FT	VR	GA	TU	RC
Gem.	Orig.	71.8	69.3	71.2	**91.2**	87.1
	Δ	−15.0	−25.8	−27.7	**−3.4**	−7.6
DS	Orig.	68.4	74.5	82.3	**91.4**	83.4
	Δ	−27.9	−31.9	−24.6	**−4.2**	+0.6
Qw.	Orig.	59.6	70.5	82.8	**88.4**	62.8
	Δ	−8.9	−13.8	−19.7	**−3.7**	−7.5

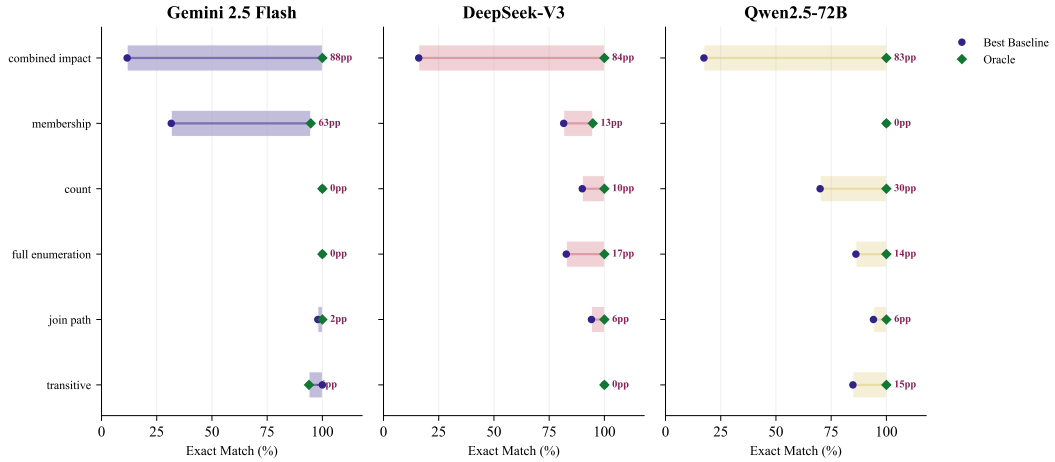

Figure 2: Unsolved subtypes: Oracle EM minus best non-oracle baseline. `combined_impact` retains >82 pp gap; `count`, `full_enumeration`, and `transitive` show 5–25 pp gaps. The remaining 9 of 13 subtypes are solved (<5 pp).

5.5 Obfuscation

As seen in Table 5, TU shows minimal degradation (Δ=−3.4% Gemini, −4.2% DeepSeek, −3.7% Qwen) while static baselines lose 9–32pp. Obfuscation replaces table names only; column names are preserved, as all baselines operate on schema-level graph topology where column semantics play no role. RC shows near-zero penalty for DeepSeek (Δ=+0.6%), confirming that code execution against the graph object is largely invariant to name perturbation.

5.6 Discussion

Our results decompose graph reasoning into **retrieval-limited** subtypes (solved by tools) and **reasoning-limited** subtypes (unsolved even with tools). The ∼35% of questions that remain unsolved by both TU and GA largely overlap: their union improves EM by only 6pp as seen in Figure 2, suggesting the bottleneck is compositional structural reasoning itself—the capability that GNN encoders are designed to provide.

`combined_impact` **specification.** Impact is the union of lineage-downstream tables and their FK-reachable neighbors: lineage transitive closure then FK BFS. This two-phase composition drives the 12% TU ceiling. RC receives five code-execution rounds (compared with three tool calls for TU), yet Gemini still scores 0% on this subtype. These findings suggest that the tool budget is not the primary bottleneck. The main difficulty appears to lie in compositional planning.

GA obfuscation penalty. The large GA Δ (−24 to −28 pp) likely reflects two compounding factors: (1) loss of semantic cues (`DimCustomer`→`Table_K`) and (2) 45% of questions exceed the 3-hop retrieval radius. Obfuscation therefore removes the schema-name cues that otherwise help LLMs infer structure beyond the retrieved fragment.

Tool tautology. TU achieving near-perfect scores on easy subtypes is by design: it isolates hard failures as genuine reasoning problems. After target-node normalization, TU reaches 100% on `membership` (largely due to Qwen's label-agnostic matching) but still only 12–13% on `combined_impact`. The limiting factor therefore does not appear to be tool availability, but compositional multi-step reasoning.

Graph baselines. While our evaluation covers standard retrieval and tool-use paradigms, we do not evaluate graph-specific agent frameworks (e.g., G-Retriever, GraphRAG, Think-on-Graph) because they typically require task-specific training data or finetuning, which violates our strict zero-shot evaluation protocol. DW-Bench establishes a standardized foundational layer for future evaluation of these hybrid graph-reasoning architectures.

5.7 Limitations

We evaluate three frontier LLMs with deterministic single-pass evaluation; broader model coverage (e.g., reasoning-optimized variants) would strengthen generality claims. Furthermore, our obfuscation protocol modifies only table names; restricting semantic leakage through column names or standardized suffixes (e.g., `_id`) is a topic for future iterations. Lastly, questions currently follow deterministic templates. While this isolates structural reasoning, future work should evaluate linguistic robustness by incorporating paraphrase variations. Additionally, we establish fixed iteration budgets (e.g., 3 calls for TU, 5 rounds for RC) and retrieval radii (3-hop) to establish baselines; extensive parametric scaling of inference compute is a topic for future study. Finally, while our lineage edges are derived from strict documentation standards, exploring LLM robustness against ambiguous or conflicting ETL provenance remains an open challenge.

Responsible use. This benchmark is intended for scientific evaluation of LLM capabilities and limitations. Researchers should not apply automated agentic tools or sandboxed code execution to unauthorized or production environments without appropriate safeguards.

Dataset licenses. TPC-DS and TPC-DI are available under the TPC EULA (free for research). OMOP CDM uses the Apache 2.0 license. AdventureWorks is released under the Microsoft Public License. Syn-Logistics is our creation, released under MIT.

6 Conclusion

DW-Bench tests whether LLMs can perform graph-topology reasoning over data warehouse schemas. We evaluate six baselines across five datasets with three LLMs, yielding 1,046 schema-level questions. Four findings are particularly notable.

(1) Tool-use dominance. Give the LLM deterministic graph tools and it reaches 87–90% micro-EM, beating the best static baseline by 7–14 pp. This indicates that LLMs are better at orchestrating graph algorithms through tool use than at performing the same reasoning internally.

(2) Triviality illusion. Easy lookup subtypes inflate micro-EM. Switch to macro-EM (equal subtype weight) and the ranking shifts: what looked like widespread success reveals significant performance disparities.

(3) Hard-task ceiling. No baseline, including agentic code-generation, exceeds 61% on hard questions; the Oracle hits >99%. One subtype, `combined_impact` (12% EM with tools), accounts for most of the gap: it requires chaining lineage traversal with FK expansion, a compositional step that current baselines handle poorly.

(4) Obfuscation invariance. Tool-Use is largely unaffected by obfuscation ($\Delta \leq 4$ pp), while static baselines decline by 9–32 pp. This result is consistent with tool-based reasoning over graph topology rather than reliance on table-name cues.

The natural next step is a **hybrid architecture**: pair interactive tools with learned graph representations (GNN encoders, graph-aware controllers) to close the compositional reasoning gap. DW-Bench supplies the per-subtype diagnostics needed to measure progress.

References

[1] P. B. Chen et al. Beaver: An enterprise benchmark for text-to-sql. In *ACL*, 2024.

[2] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[3] D. Edge, H. Trinh, N. Cheng, et al. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[4] B. Fatemi, J. Halcrow, and B. Perozzi. Talk like a graph: Encoding graphs for large language models. In *ICLR*, 2024.

[5] M. Fey and J. E. Lenssen. Fast graph representation learning with pytorch geometric. *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[6] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using networkx. SciPy Conference, 2008.

[7] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *NeurIPS*, 2024.

[8] G. Hripcsak et al. Observational health data sciences and informatics (OHDSI): Opportunities for observational researchers. *Studies in Health Technology and Informatics*, 216, 2015.

[9] B. Jin, C. Xie, J. Zhang, K. K. R. Meng, H. Zhang, S. Zhang, D. Bo, et al. Graph chain-of-thought: Augmenting large language models by reasoning on graphs. *Findings of ACL*, 2024.

[10] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.

[11] F. Lei et al. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. In *ICLR*, 2025.

[12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

[13] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K. Chang, F. Si, and Y. Li. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. In *NeurIPS*, 2023.

[14] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[15] N. Reimers and I. Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *EMNLP*, 2019.

[16] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, H.-Y. Shum, and J. Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *ICLR*, 2024.

[17] J. Tang et al. Grapharena: Benchmarking large language models on graph computational problems. In *ICLR*, 2025.

[18] Z. Tang, Y. Zhang, S. Cai, and R. Wang. Llm-fk: Multi-agent llm reasoning for foreign key detection in large-scale complex databases. *arXiv preprint arXiv:2603.07278*, 2026.

[19] G. Team et al. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2024.

[20] H. Wang, S. Feng, T. He, Z. Tan, X. Han, and Y. Tsvetkov. Nlgraph: Can llms solve graph problems in natural language? In *NeurIPS*, 2023.

[21] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *EMNLP*, 2018.

[22] Z. Yuan et al. Gracore: Benchmarking graph comprehension and complex reasoning in large language models. In *COLING*, 2025.

[23] Y. Zhao, C. Zhao, L. Nan, Z. Qi, W. Zhang, X. Tang, B. Mi, and D. Radev. Robut: A systematic study of table qa robustness against human-annotated adversarial perturbations. In *ACL*, 2023.

A Supplementary Figures

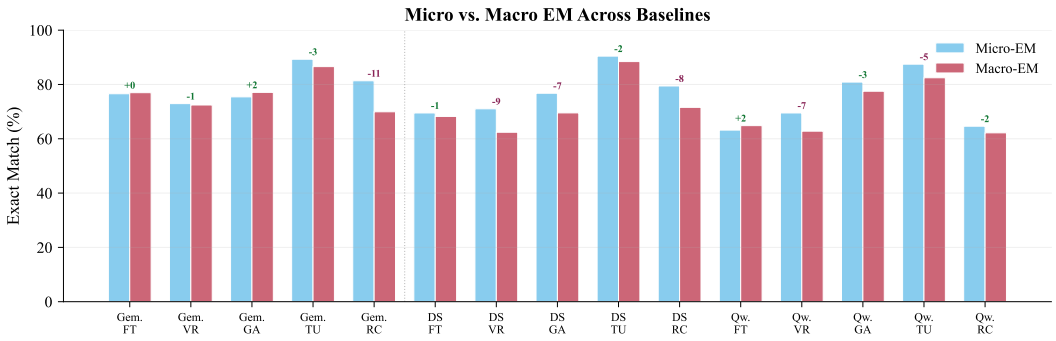

Figure 3: The Triviality Illusion: Micro vs. Macro EM. Negative deltas reveal that easy subtypes inflate aggregate scores for static baselines.

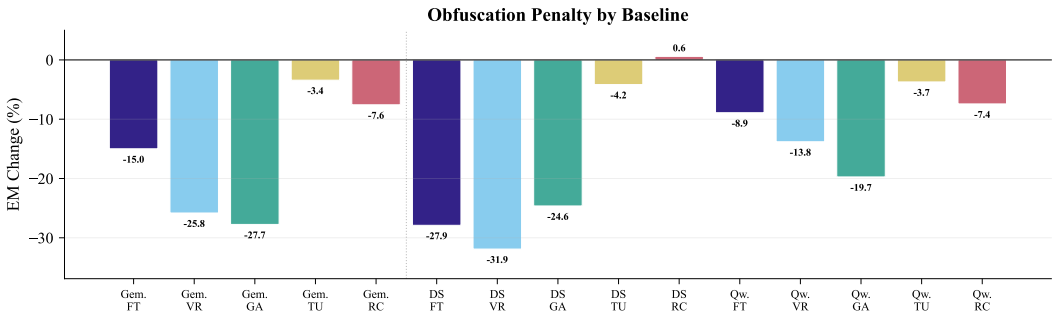

Figure 4: Obfuscation penalty by baseline and model. Tool-Use loses only 3–4% while static baselines lose 9–32%.

B `combined_impact`: Formal Specification

The `combined_impact` subtype captures the set of tables transitively affected when a source table is modified, propagating first through lineage then through FK dependencies.

Algorithm 1 `combined_impact`(G, t)

Require: Hetero-graph $G = (V, E_{fk} \cup E_{lin})$; source table $t \in V$
Ensure: Set of affected tables $A \subseteq V$

```
L ← LineageClosure(G, t)          ▷ Directed BFS over E_lin from t
A ← L
for each u ∈ L do
    F_u ← FkBFS(G, u)             ▷ Undirected BFS over E_fk from u
    A ← A ∪ F_u
end for
A ← A \ {t}                       ▷ Exclude source table itself
return A
```

Corner cases. (1) Lineage DAGs contain no cycles by construction; future datasets with relaxed constraints should detect and skip back-edges. (2) Duplicates during union are naturally eliminated by set semantics. (3) FK BFS treats edges as undirected (bidirectional), consistent with the connectivity definition for all Silo-category questions.

C Hop-Distance Analysis

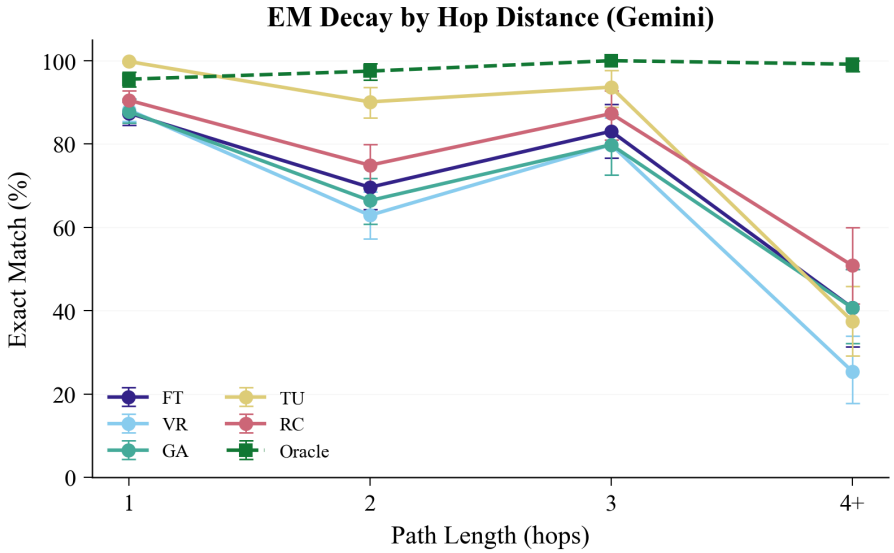

Figure 5: EM by gold path length. Static baselines collapse beyond 3 hops; Tool-Use degrades more gracefully.

Table 6: Minimum evidence hops per subtype (Syn-Logistics). 45.4% of questions exceed the GA 3-hop BFS radius.

Subtype	n	Exceeds 3h	%	GA EM
count	26	26	100%	100%
membership	20	20	100%	0%
combined_impact	34	34	100%	0%
full_enum	25	25	100%	100%
connected	25	25	100%	100%
isolation	25	25	100%	100%
transitive	29	4	14%	93%
hop_count	44	5	11%	100%
direct_fk	36	3	8%	100%
join_path	26	2	8%	81%
multi_source	42	0	0%	95%
forward	20	0	0%	100%
reverse	20	0	0%	100%
Total	372	169	**45.4%**	83.1%

D Dataset Details

Syn-Logistics Design. The Syn-Logistics dataset guarantees $n \geq 20$ per subtype via three parallel 4-hop lineage chains across five components. Table names are drawn from realistic supply-chain terminology (64 tables, 96 FK, 35 lineage).

Lineage Edge Curation. **AW**: Microsoft reference docs (39 edges). **TPC-DS**: No lineage (pure FK). **TPC-DI**: Official spec ETL flows (21 edges). **OMOP**: OHDSI ETL conventions (21 edges). **Syn-Log**: By construction (35 edges).

E Evaluation Details

Models. **Gemini 2.5 Flash** (Google, closed-weight, accessed via API) and **DeepSeek-V3** (DeepSeek, 671B MoE, MIT licensed, accessed via API). Both use greedy decoding (τ=0, p=1.0) with a single deterministic pass per question for exact reproducibility.

Scoring. *Exact Match (EM)*: predicted answer must exactly match the gold answer after canonicalization. *Path validation*: for `join_path`, alternative shortest paths of equal length are validated against the FK adjacency matrix. *List canonicalization*: lists are lowercased and sorted alphabetically before comparison; order does not matter. *F1*: token-level overlap between predicted and gold answer sets.

Retrieval. Vector-RAG uses `all-MiniLM-L6-v2` [15] embeddings with FAISS exact search and k=15 retrieved chunks. Graph-Augmented uses 3-hop BFS neighborhoods from mentioned tables. Tool-Use provides 9 graph algorithm tools (`shortest_path`, `connected_components`, `get_component_of`, `get_fk_neighbors`, `get_lineage_forward`, `get_lineage_reverse`, `transitive_lineage`, `check_fk_adjacency`, `list_tables`) with up to 3 calls per question. ReAct-Code provides a Python REPL with NetworkX and allows up to 5 code execution rounds per question.

Statistical treatment. All figures include 95% bootstrap confidence intervals computed via 2,000 resamples over questions. Per-question results (JSON) are released for community bootstrap and stratification analyses.

F Example Questions and Predictions

Table 7: Representative questions across difficulty levels and categories, with gold answers and Tool-Use (TU) predictions. ✓ = correct, ✗ = incorrect. Tool-Use succeeds on lookup-style questions (direct FK, shortest path) but fails on exhaustive enumeration (membership) and multi-hop composition (combined impact).

Subtype	**Question**	**Gold**	**TU Pred.**
`direct_fk` *Easy*	Is there a direct FK between `DimProduct` and `FactInternetSales`?	yes	yes ✓
`join_path` *Medium*	Shortest FK path from `DimCustomer` to `DimProduct`?	[DimCustomer, FactInternetSales, DimProduct]	[DimCustomer, FactInternetSales, DimProduct] ✓
`membership` *Hard*	Which component contains `DimCurrency`? List all tables.	[DimCurrency, FactCurrencyRate, DimDate, . . .] (12)	[DimCurrency] ✗
`combined_impact` *Hard*	Which tables are transitively affected (lineage+FK) if `raw_purchase_orders` is modified?	[stg_purchase_orders, dim_supplier, . . .] (8)	[stg_purchase_orders] ✗

G Partial Correctness (F1 Scores)

For list-typed target answers (e.g., set enumeration tasks), Exact Match (EM) acts as a strict step-function. To better reflect partial correctness, we report the token-level F1 scores for the three primary enumeration subtypes: `membership`, `full_enumeration`, and `combined_impact` (Table 8). F1 scores demonstrate that models retrieve partially correct answers even when failing EM, though the compositional gap remains severe on `combined_impact` (F1 peaks at 45.4% vs. 100% for Oracle).

Table 8: F1 scores (%) for enumeration subtypes across models and baselines.

Subtype	Model	FT	VR	GA	TU	RC
`membership`	Gem.	0.0	0.0	0.0	58.7	0.0
	DS	0.0	0.0	9.4	90.1	0.0
	Qw.	75.0	53.6	94.6	**100.0**	**100.0**
`full_enum`	Gem.	**100.0**	79.4	**100.0**	94.7	69.8
	DS	90.5	70.0	**91.2**	90.4	77.2
	Qw.	88.6	67.8	**92.5**	66.8	39.5
`combined_impact`	Gem.	17.6	11.2	11.1	**32.9**	4.3
	DS	23.9	22.1	14.2	**31.5**	9.1
	Qw.	20.8	19.3	15.0	44.8	**45.4**